

Data Structure

This page has been generated by [markdown2](#) package.

For each URL, we collect various features and raw data related to active and passive scan of the submitted URL and its features. The general structure of the data is as follows:

The format of the data is JSON. So after downloading the data, first we need to decode it.

Python3 sample code:

```
import json
json.loads(data)
```

Bash sample code:

```
cat data | jq
```

The decoded data has the following fields:

- data (object)
 - [metadata](#) (object)
 - *rd* (string): registered domain part of the URL
 - *fqdn* (string): fully-qualified domain name of the URL
 - *tld* (string): Top-level domain (TLD) of the underlying domain name of the URL
 - *src* (string): source of the URL
 - *trg* (string): target of the URL (in case of phishing or malware) e.g., *paypal*
 - *dt* (string): URL submission date in yyyy-mm-dd format
 - *clk* (string): URL submission time in HH:MM:SS format
 - *discovery_time* (integer): URL submission time in Unix timestamp format
 - *sid* (string): Submission ID for this URL. This is a unique value and the key to access this URL in our DB.
 - *fref*(string): File reference of this URL. This value is used internally by the system. You can use this field to permanently download data from our server.
 - *url* (string): The submitted URL
 - *hashtags* (list/array): an array of keywords for this URL e.g., ["#malware","#dga","#sample"]
 - [tldindex](#) (object): The object specifying the TLD maliciousness index from Spamhaus
 - [special_rd](#) (Array/list): if the domain is special e.g., free subdomain service or dynamic DNS provider. Each member of the array/list is an object with the following values (more on [special domains](#)).
 - *rd* (string): registered domain
 - *dt* (string): date of database update in yyyy-mm-dd format.
 - *domain_type* (string): short name of the domain type e.g., "fsp" for "free subdomain providers" (more info [here](#))
 - *extra_info* (string): long description of the domain_type.
 - *input_metadata* (JSON object): showing extra information/description about the URL. This can be sent by the user at the time of URL submission through the API.
 - [host_info](#) (object): all the information related to the host and DNS resource record for the submitted URL.
 - *url* (string): The submitted URL by the user either through web or API.
 - [content_info](#) (object): all the information regarding active content of the submitted URL, har logs and screenshots.
 - [tech_info](#) (object): Technologies used in the html page of the submitted URL based on [Wappalyzer](#) list.
 - **has_path** (boolean): This specifies if the submitted URL has path or not. Anything after the registered domain can be considered as *path* for a URL. For example: "http://google.com" does not have path while "http://google.com/data" or "http://google.com/?q=search" or "http://google.com/#frag" (with fragment) all have path.
 - **subdomain** (string/null): If the submitted URL has a subdomain, then this value contains the subdomain otherwise it's null.
 - *has_subdomain* (boolean): If the submitted URL has a subdomain (not considering www), then this value is true otherwise it's false.
 - [additional](#) (object): This object contains additional information about the underlying domain and/or possible subdomain of the submitted URL. It is always available and always has two members:
 - [rd](#) (object): This object contains information about the [ssl info](#), , [wayback info](#), [content info](#), [host info](#) and [tech info](#) for the underlying registered domain of the submitted URL.

- [sd](#) (object): This object is the same as [rd](#) but contains all the information related to the possible subdomain (if there exists) of the submitted URL.

Metadata Object

Whenever a URL submitted to our system, we extract some information from the URL itself, like the underlying registered domain, fully-qualified domain name (in case of subdomains) and TLD of the domain name. Some other data are also stored in this object to make it easier to query some specific data.

Spamhaus Index

This is the index which is shown by [Spamhaus](#) website for each TLD. This index value may change every day based on the number of domains for that specific TLD and the number of maliciously registered domains. For example, as of 2020-11-16, the Spamhaus index for *.tk* domains is 3.12 while this value for *.com* domains is 0.44. This means that generally, *.tk* domains are used more in malicious activity compared to *.com* domains (considering the size of zone files).

We store this index on a daily basis in our database and for each submitted URL, you can access the value of index by querying **metadata->tld_index**.

Special Domains

Some domain names are in the category of special domains. This means that these domains are not malicious by themselves but are (ab)used by malicious actors.

As an example of these domains are *subdomain providers* companies like *appspot.com*. As of 2020-11-16, we scanned 6,613 URLs related to subdomains of *appspot.com*.

Apart from *free subdomain providers*, there are *dynamic DNS providers* and *URL shortener systems*.

To check if the domain is special, you can use the following Python3.x code:

```
import json
# 'data' is the json data for the scanned URL
data = json.loads(data)
metadata = data.get("metadata",{})
success = metadata.get("special_rd",[])
for i in special_rd:
    domain_type = i.get("domain_type","")
    extra_info = i.get("extra_info","")
    rd = i.get("rd","")
    dt = i.get("dt","")
    print("{} is {}({}) scan date: {}".format(rd,extra_info,domain_type,dt))
# end for
```

One domain name can be both *subdomain provider* and *dynamic DNS provider*.

additional object

additional object contains all the information related to the underlying registered domain and possible subdomain of the submitted URL. It contains two objects:

- rd (object)
- sd (object)

Each *rd* and *sd* object contains the following information:

- [host_info](#)
- [ssl_info](#)
- [content_info](#)
- [tech_info](#)
- [wayback_info](#)

Depending on the type of the submitted URL, some data might not be available. For example assuming the following submissions:

1 - <http://domain.com/somepath.php>

- The *path* part of the above mentioned URL is *somepath.php* and therefore, `metadata->has_path` is true.
- No subdomain is available for this URL. Therefore, `metadata->subdomain` is null/None.
- `content_info` object contains the content features of the *somepath.php* page.

- `host_info` object contains the host and DNS information of `domain.com` domain.
- `tech_info` object contains the web technologies used in designing `somepath.php` page.
- `additional->rd->content_info` object contains the content features of the homepage of the `domain.com` by visiting "`http://domain.com`" separately.
- `additional->rd->tech_info` object contains the technologies used in designing the homepage of the `domain.com` page.
- `additional->rd->host_info` object is the same as `host_info` object since the submitted URL uses the same DNS info of the registered domain (because there is no subdomain involved).

2 - `http://mysubdomain.domain.com/somepath.php`

- The *path* part of the above mentioned URL is *somepath.php* and therefore, `metadata->has_path` is true.
- subdomain is available for this URL. Therefore, `metadata->subdomain` is "`mysubdomain`".
- `metadata->has_subdomain` is true
- `content_info` object contains the content features of the `somepath.php` page.
- `host_info` object contains the host and DNS information of `mysubdomain.domain.com`.
- `tech_info` object contains the web technologies used in designing `somepath.php` page.
- `additional->rd->content_info` object contains the content features of the homepage of the `domain.com` by visiting "`http://domain.com`" separately.
- `additional->rd->tech_info` object contains the technologies used in designing the homepage of the `domain.com` page.
- `additional->rd->host_info` object contains the host and DNS information of the `domain.com` which is different from the host and DNS information of the `mysubdomain.domain.com`.
- `additional->sd->host_info` object contains the host and DNS information of the `mysubdomain.domain.com` which is the same info as `host_info` object.

The structure of each object explained in the following sections.

Host Info

host_info object contains all the information about the DNS resource records, SSL/TLS info as well as IP-location info of the registered domain or subdomain. The structure of the `host_info` object is as follows:

- `host_info` object
 - **a** (object): Information related to the "A" record of the registered domain
 - `status` (string): Possible values are:
 - NOERROR: the DNS query was successful
 - ERROR: the DNS query was NOT successful
 - TIMEOUT: timeout happened before successfully querying the record.
 - NXDOMAIN: The domain does not exist or DNS query does not exist.
 - EXCEPTION: The packet is malformed or something is wrong in the query.
 - NOANSWER: No answer received from the server.
 - `answers` (array/list): list of records found in the answer section of the DNS query.
 - **mx** (object): Information related to the "MX" record of the registered domain
 - `status` (string): "NOERROR", "ERROR", "TIMEOUT", "NXDOMAIN", "EXCEPTION" and "NOANSWER".
 - `answers` (array/list): list of records found in the answer section of the DNS query.
 - **txt** (object): Information related to the "TXT" record of the registered domain
 - `status` (string): "NOERROR", "ERROR", "TIMEOUT", "NXDOMAIN", "EXCEPTION" and "NOANSWER".
 - `answers` (array/list): list of records found in the answer section of the DNS query.
 - **aaaa** (object): Information related to the "AAAA" record of the registered domain
 - `status` (string): "NOERROR", "ERROR", "TIMEOUT", "NXDOMAIN", "EXCEPTION" and "NOANSWER".
 - `answers` (array/list): list of records found in the answer section of the DNS query.
 - **ns** (object): Information related to the "NS" record of the registered domain
 - `status` (string): "NOERROR", "ERROR", "TIMEOUT", "NXDOMAIN", "EXCEPTION" and "NOANSWER".
 - `answers` (array/list): list of records found in the answer section of the DNS query.
 - **dmARC** (object): Information related to the "DMARC" record of the registered domain
 - `status` (string): "NOERROR", "ERROR", "TIMEOUT", "NXDOMAIN", "EXCEPTION" and "NOANSWER".
 - `answers` (array/list): list of records found in the answer section of the DNS query.
 - **soa** (object): Information related to the "SOA" record of the registered domain
 - `status` (string): "NOERROR", "ERROR", "TIMEOUT", "NXDOMAIN", "EXCEPTION" and "NOANSWER".
 - `answers` (array/list): list of records found in the answer section of the DNS query.

- **maxmind** (array/list of objects): information related to the autonomous system (AS) number. Each object has the following structure:
 - answers (object):
 - cc_code (string): 2-letter country code related to the A record of the domain
 - asn_code (string): AS number related to the A record of the domain
 - asn_org (string): AS organization related to the A record of the domain
 - ip (string): A record of the domain extracted from *host_info->domain->a->answers*
- **ssl** (object):
 - issuer (string): Certificate authority which issued the SSL/TLS certificate
 - valid_from (string): *valid_from* field of the SSL/TLS certificate.
 - valid_until (string): *valid_until* field of the SSL/TLS certificate.
 - subject (array/list): list of all the common names in this certificate.
 - is_valid_cert (boolean): if the certificate is valid for this domain (e.g., not self-signed)

For example:

1. Printing all the A records of the host_info object.

```
import json
# 'data' is the downloaded result from the server
data = json.loads(data)
host_info = data.get("host_info", {})
a_obj = host_info.get("a", {})
status = a_obj.get("status")
answers = a_obj.get("answers", [])
print("The status of the DNS A record is {}".format(status))
for x in answers:
    print("The A record is: {}".format(x))
# end for
```

2. Get the country code for each A record.

```
```python
import json
'data' is the downloaded result from the server
data = json.loads(data)
host_info = data.get("host_info", {})
maxmind = host_info.get("maxmind", [])
for obj in maxmind:
 answers = obj.get("answers", {})
 ip = answers.get("ip")
 cc = answers.get("cc_code")
 asn_code = answers.get("asn_code")
 asn_org = answers.get("asn_org")
 msg = "IP {} in {} with AS {} and organization {}".format(ip, cc, asn_code, asn_org)
 print(msg)
end for-loop
```

## Content Info

**content\_info** object contains all the information about the content as well as screenshots of the record (could be the submitted URL and/or the registered\_domain and/or the subdomain).

The structure of the *content\_info* object is as follows:

- **content\_info**:
  - **title** (string): title of the page
  - **html** (string): HTML content of the page
  - **extra\_data** (object): extra information about the page
  - **har** (array/list): array of objects with the following fields:
    - **request** (object): contains the following fields
      - **id** (string): unique identifier of the request
      - **url** (string): requested URL
    - **response** (object): contains the following fields
      - **id** (string): identifier match with the **request->id**
      - **protocol** (string): protocol of the request



- **remotePort** (string): remote port number the request sent to
- **remoteIp** (string): remote IP address the request sent to
- **status** (integer): response status (e.g., 200 means success)
- **statusText** (string): description of the *status* field.
- **contentType** (string): mime type of the response
- **headers** (object): mostly contains HTTP response headers in a form of key-value list.
- **screenshot** (object): an object contains the base64 encoded string of the screenshot of the page in JPG format.
- **cookies** (array/list): array of object, each representing the cookie (key-value) set.
- **responses** (array/list): array of objects, each with the following fields
  - **requestId** (string): unique identifier matches with on of the *har->request->id*
  - **file\_type** (string): type of the file sent from the server based on Linux [file](#) command.
  - **md5** (string): MD5 hash of the downloaded file
  - **ssdeep** (string): SSDEEP hash of the downloaded file

## Notes

- Each field of the *content\_info* object may or may not exist. This means that you need to check if the field is there and then get its content.

## Har, cookies and responses objects

*har* object is part of the *content\_info* object. To better understand the *har* object, let's take a look at the following example:

If you try to visit <http://icann.org> using a browser, the browser will send some extra requests to download other dependencies like .CSS files, .JS files and possibly a few images, favicons and web trackers like this:

- <https://www.icann.org/>
- [https://www.icann.org/uploads/featured\\_item/image/1/thumb\\_Maarten\\_blog10\\_icannorg\\_492x279\\_1.jpg](https://www.icann.org/uploads/featured_item/image/1/thumb_Maarten_blog10_icannorg_492x279_1.jpg)
- [https://www.icann.org/uploads/featured\\_item/image/2/thumb\\_New\\_IP\\_Publication\\_01\\_icannorg\\_492x279.png](https://www.icann.org/uploads/featured_item/image/2/thumb_New_IP_Publication_01_icannorg_492x279.png)
- <https://www.recaptcha.net/recaptcha/api.js>
- [https://www.icann.org/glossary\\_terms.js](https://www.icann.org/glossary_terms.js)
- <https://www.icann.org/favicon.ico>
- ...

*har* object in the *content\_info*, is an array of all the requests and responses related to the URL you want to visit.

*cookies* object is an array of all the cookies set after visiting <http://icann.org> URL.

*responses* object is an array of objects containing the MD5 and SSDEEP hashes of all the resources downloaded by browser (sent by the server).

## SSL Info

The *ssl\_info* object contains all the historical issued SSL/TLS certificate for the domain name of the submitted URL. For some old domains, it's possible to have more than 200 records and for some domains without SSL, we may have an empty object.

The structure is as follows:

- *ssl\_info* (object)
  - *common\_name* (string): common name field of the certificate
  - *entry\_timestamp* (string): timestamp field of the certificate
  - *id* (integer): id field of the certificate
  - *issuer\_name* (string): certificate issuer name
  - *name\_value* (string): raw list of domains and subdomains the certificate issued for (separated by newline character).
  - *serial\_number* (string): serial number of the certificate
  - *issuer\_ca\_id* (integer): ID of the issuer
  - *not\_before* (string): start of validity period of the certificate
  - *not\_after* (string): end of validity period of the certificate

## Wayback Info

One way to check the history of the domains as well as their reputation is to check the Internet Archive. Using Archive system, you can check the older website pages of the domain and also you can check how old the website is (the domain could be very old but the website of the domain could be new or vice versa in case of dropcatching).

The structure of the *wayback\_info* object is as follows:

- *wayback\_info* (object)
  - *first\_ts* (string): The first time domain appeared in Archive system, format is 'yyyymmddHHMMSS'

- last\_ts (string): The last time domain appeared in Archive systems, format is 'yyyymmddHHMMSS'
- years (key-value object): the 'keys' are years and the values are arrays of 12 integers each shows the number of appearance in one month.

Example:

```
{
 "last_ts": "20200923002448",
 "years": {
 "2013": [
 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0
],
 "2014": [
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1
],
 "2020": [
 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0
]
 },
 "first_ts": "20130411144706"
}
```

In the above example, the domain name appeared in Archive system for 3 years (2013,2014 and 2020) one time in April 2013, one time in December 2014 and 2 times in August and September 2020. The first ans last seen also shown by *first\_ts* and *last\_ts* fields.

## Tech Info

The *tech\_info* object contains all the information we were able to extract about the technologies, libraries and frameworks used in the submitted URL, homepage of the domain name or homepage of the without subdomain.

The structure of the object is as follows:

- tech\_info (object)

array of strings, each contains the name of the technology.

All the technology fingerprinting algorithm is based on the [Wappalyzer](#) signature list.

Below you can see the list of technology for scanning a URL with subdomain:

TECHNOLOGY INFO (DOMAIN, SUBDOMAIN, URL) ×

URL Tech: ⓘ

MySQL

Font Awesome

Modernizr

TweenMax

WordPress

WooCommerce

Google Analytics

php PHP

Yoast SEO

Google Maps

Google Font API

jQuery

Domain Tech: ⓘ

MySQL

TweenMax

Font Awesome

WordPress

WooCommerce

Google Analytics

php PHP

Yoast SEO

Modernizr

Google Font API

jQuery

Subdomain Tech:

Nothing found!!